ASD
Unit-IV
Agile Requirements

USER STORIES-

User stories are a key component of agile software development. They are short, simple descriptions of a feature or functionality from the perspective of a user. User stories are used to capture requirements in an agile project and help the development team understand the needs and expectations of the users.
Here are some key characteristics of user stories:

User-centric: User stories focus on the needs of the user and what they want to achieve with the software.
Simple: User stories are short and simple descriptions of a feature or functionality.
Independent: User stories can stand on their own and do not rely on other user stories.
Negotiable: User stories are open to discussion and can be refined and modified based on feedback from stakeholders.
Valuable: User stories provide value to the user and the business.
Estimable: User stories can be estimated in terms of time and effort required for implementation.
Testable: User stories can be tested to ensure they meet the needs of the user.
Prioritized: User stories are prioritized based on their importance to the user and the business goals.
Iterative: User stories are developed iteratively, allowing for feedback and changes throughout the development process.

Consistent: User stories follow a consistent format, making them easy to understand and work with.

Contextual: User stories are written in a way that provides context to the development team, helping them understand the user's needs and goals.

Acceptance criteria: User stories have clear and specific acceptance criteria that define when the story is considered "done" and ready for release.

Role-based: User stories are written from the perspective of a specific user role, helping to ensure that the development team is building features that are relevant and useful to that user.

Traceable: User stories are tracked and linked to specific features and functionality in the software, making it easy to trace back to the original user need.

In agile software development, user stories are typically written on index cards or in a digital format, and are used to drive the development process. The development team uses user stories to plan and prioritize work, estimate the effort required for implementation, and track progress towards completing the user stories.

By using user stories in agile software development, teams can ensure that they are building software that meets the needs of the users and delivers value to the business.

Pattern of User Story:

User stories are completely from the end-user perspective which follows the Role-Feature-Benefit pattern.

```
As a [ type of user ], I want [ an action ], so that [ some reason ]
```

For example :

As the project manager of a construction team, I want our team-messaging app to include file sharing and information update so that my team can collaborate and communicate with each other in real-time as a result the construction project development and completion will be fast.

Writing User Stories :

User stories are from a user perspective. So when user stories are written, users are given more importance during the process. Some points outlined which are taken into consideration during writing user stories like

1. Requirements
2. Tasks and their subtasks
3. Actual user
4. Importance to user words/feedback
5. Breaking user stories for larger requirements

With this also some other principles which are given importance during creating user stories are discussed below.

INVEST Principle of User story :

A good user story should be based on INVEST principle which expresses the quality of the user story because in base a good software product is completely dependent upon a good user story. In 2003 INVEST checklist was introduced by Bill Wake in an article.

1. **Independent -**
   Not dependent on other.
2. **Negotiable -**
   Includes the important avoid contract.
3. **Valuable -**
   Provide value to customer.
4. **Estimable -**
   It should be estimated.
5. **Small -**
   It should be simple and small not complex.
6. **Testable -**
   It should be evaluated by pre-written acceptance criteria.

3 C's in User Stories :

1. **Card -**
   Write stories on cards, prioritize, estimate and schedule it accordingly.
2. **Conversation -**
   Conduct conversations, Specify the requirements and bring clarity.
3. **Confirmation -**
   Meet the acceptance criteria of the software.

Working with User Stories :

Below points represents working with user stories

1. Once the user story is fully written then it undergoes review and verification.
2. In project workflow meetings it is reviewed and verified then added to the actual workflow.
3. Actual requirements and functionality are decided based on the stories.
4. User stories are scored based on their complexity and the team starts work based on user stories.

Advantages of using User Stories in Agile Software Development:

1. User stories help to keep the focus on the user's needs and expectations, which leads to better customer satisfaction.
2. User stories are easy to understand and can be created quickly, which speeds up the requirements gathering process.
3. User stories are flexible and can be refined and modified easily as requirements change.
4. User stories are independent, which makes it easier to prioritize and plan the work.
5. User stories are small and manageable, which makes it easier to estimate effort and track progress.
6. User stories promote collaboration between stakeholders, which leads to better communication and understanding.
7. User stories help to reduce scope creep and feature bloat, as they focus on the essential needs of the user.

Disadvantages of using User Stories in Agile Software Development:

1. User stories may not provide enough detail or clarity to fully capture the requirements.
2. User stories may not be sufficient for complex or large-scale projects.
3. User stories may be subjective and influenced by the biases of the stakeholders.
4. User stories may not capture all of the requirements, which can lead to gaps in the software.
5. User stories may not be suitable for projects with a high degree of technical complexity.
6. User stories may not capture non-functional requirements, such as performance, scalability, or security, which are critical to the success of the software.
7. User stories may not be appropriate for projects with a high degree of interdependence between features or components, as they are designed to be independent.
8. User stories may not be suitable for teams that lack experience with Agile methodologies or have difficulty working collaboratively.

Backlog Management-

Backlog management in Agile is the continuous process of creating, refining, and prioritizing a dynamic list of tasks—mainly user stories—to ensure the team focuses on high-value features. Managed primarily by the Product Owner, it involves regular grooming (refinement) sessions to break down complex tasks, estimate efforts, and align with stakeholder needs. This keeps the product backlog lean and actionable.
Key Components and Practices:

Product Backlog: A master list of all desired features, improvements, and fixes.

Sprint Backlog: A subset of top-priority items selected for the current development iteration, owned by the development team.

Backlog Refinement (Grooming): Regularly updating the backlog to remove obsolete items, add new ones, and refine existing stories with clear acceptance criteria.

Prioritization Techniques: Using methods like MoSCoW (Must-have, Should-have, Could-have, Won't-have) or RICE (Reach, Impact, Confidence, Effort) to rank items by business value.

Tools: Using tools like Jira or Trello to maintain visibility.

Benefits of Effective Backlog Management:

Improved Efficiency: Keeps the team focused on the most valuable tasks, reducing wasted effort.

Better Adaptability: Allows teams to quickly pivot based on changing requirements or customer feedback.

Enhanced Quality: Regular refinement ensures technical requirements are understood before development begins, improving product quality.

Reduced Scope Creep: A well-managed, prioritized list prevents the backlog from becoming unmanageably large.

Agile Architecture: Feature Driven Development

An Agile methodology for developing software, Feature-Driven Development (FDD) is customer-centric, iterative, and incremental, with the goal of delivering tangible software results often and efficiently. FDD in Agile encourages status reporting at all levels, which helps to track progress and results.

FDD allows teams to update the project regularly and identify errors quickly. Plus, clients can be provided with information and substantial results at any time. FDD is a favorite method among development teams because it helps reduce two known morale-killers in the development world: Confusion and rework./p>

First applied in 1997 during a project for a Singapore bank, FDD was developed and refined by Jeff De Luca, Peter Coad and others. The original project took 15 months with 50 people, and it worked; it was followed by a second, 18-month long, 250-person project./p>

How is FDD Different from Scrum?

FDD is related to Scrum, but as its name implies, it's a feature-focused method (as opposed to a delivery-focused method). Features are a foundational piece of FDD; they're to FDD what user stories are to Scrum: Small functions that are, most importantly, client-valued.

> *"During FDD, a feature should be delivered every 2-10 days – which differs from Scrum, in which sprints typically last two, but sometimes four, weeks."*

FDD values documentation more than other methods (Scrum and XP included), which also creates differences in the roles of meetings. In Scrum, teams typically meet on a daily basis; in FDD, teams rely on documentation to communicate important information, and thus don't usually meet as frequently.

Another key difference is the end user; in FDD, the actual user is viewed as the end user, whereas in Scrum it's typically the Product Owner who is seen as the end user.

How Does FDD Work?

Typically used in large-scale development projects, five basic activities exist during FDD:

- Develop overall model
- Build feature list
- Plan by feature
- Design by feature
- Build by feature

An overall model shape is formed during the first two steps, while the final three are repeated for each feature. The majority (roughly 75%) of effort during FDD will be spent on the fourth and fifth steps – Design by Feature and Build by Feature.

> *Teams using all Agile methodologies operate with the primary goal of quickly and effectively satisfying the needs of their customers; FDD is no exception.*

However, the difference is that once a goal has been identified, teams following FDD organize their activities by features, rather than by project milestones or other indicators of progress.

How is a Feature Defined?

In FDD, each feature is useful and important to the client and results in something tangible to showcase. And because businesses appreciate quick results, the methodology depends on its two-week cycle.

Stages of Feature-Driven Development

Stage 0: Gather Data

As with all Agile methodologies, the first step in FDD is to gain an accurate understanding of conten and context of the project, and to develop a clear, shared understanding of the target audience and their needs. During this time, teams should aim to learn everything they can about the why, the what, and the for whom about the project they're about to begin (the next few steps will help clarify the how). This data-gathering can be thought of as stage 0, but one that cannot be skipped. To compare product development with writing a research paper, this is the research and thesis development step.

Once teams have a clear understanding of their goals, the targeted audience and their current (and potentially, future) needs, the first named stage in FDD can begin: Developing an Overall Model.

Develop an overall model

Continuing the research paper metaphor, this stage is when the outline is drafted. Using the “thesis” (aka primary goal) as a guide, the team will develop detailed domain models, which will then be merged into one overall model that acts as a rough outline of the system. As it develops and as the team learns, details will be added.

Build a features list

Use the information assembled in the first step to create a list of the required features. Remember, a feature is a client-valued output. Make a list of features (that can be completed in two weeks’ time), and keep in mind that these features should be purposes or smaller goals, rather than tasks.

Plan by Feature

Enter: Tasks. Analyze the complexity of each feature and plan tasks that are related for team members to accomplish. During the planning stage, all members of the team should take part in the evaluation of features with the perspective of each development stage in mind. Then, use the assessment of complexity to determine the order in which each feature will implemented, as well as the team members that will be assigned to each feature set.

This stage should also identify class owners, individual developers who are assigned to classes. Because every class of the developing feature belongs to a specific developer, someone is responsible for the conceptual principles of that class, and should changes be required to multiple classes, then collaboration is necessary between the owners of each to implement them.

And while class owners are important to FDD, so are feature teams. In feature teams, specific roles are defined, and a variety of viewpoints are encouraged. This ensures that design decisions consider multiple thoughts and perspectives.

Design by Feature

A chief programmer will determine the feature that will be designed and build. He or she will also determine the class owners and feature teams involved, while defining the feature priorities. Part of the group might be working on technical design, while others work on framework. By the end of the design stage, a design review is completed by the whole team before moving forward.

Build by Feature

This step implements all the necessary items that will support the design. Here, user interfaces are built, as are components detailed in the technical design, and a feature prototype is created. The unit is tested, inspected and approved, then the completed feature can be promoted to the main build. Any feature that requires longer time than two weeks to design and build is further broken into features until it meets the two-week rule.

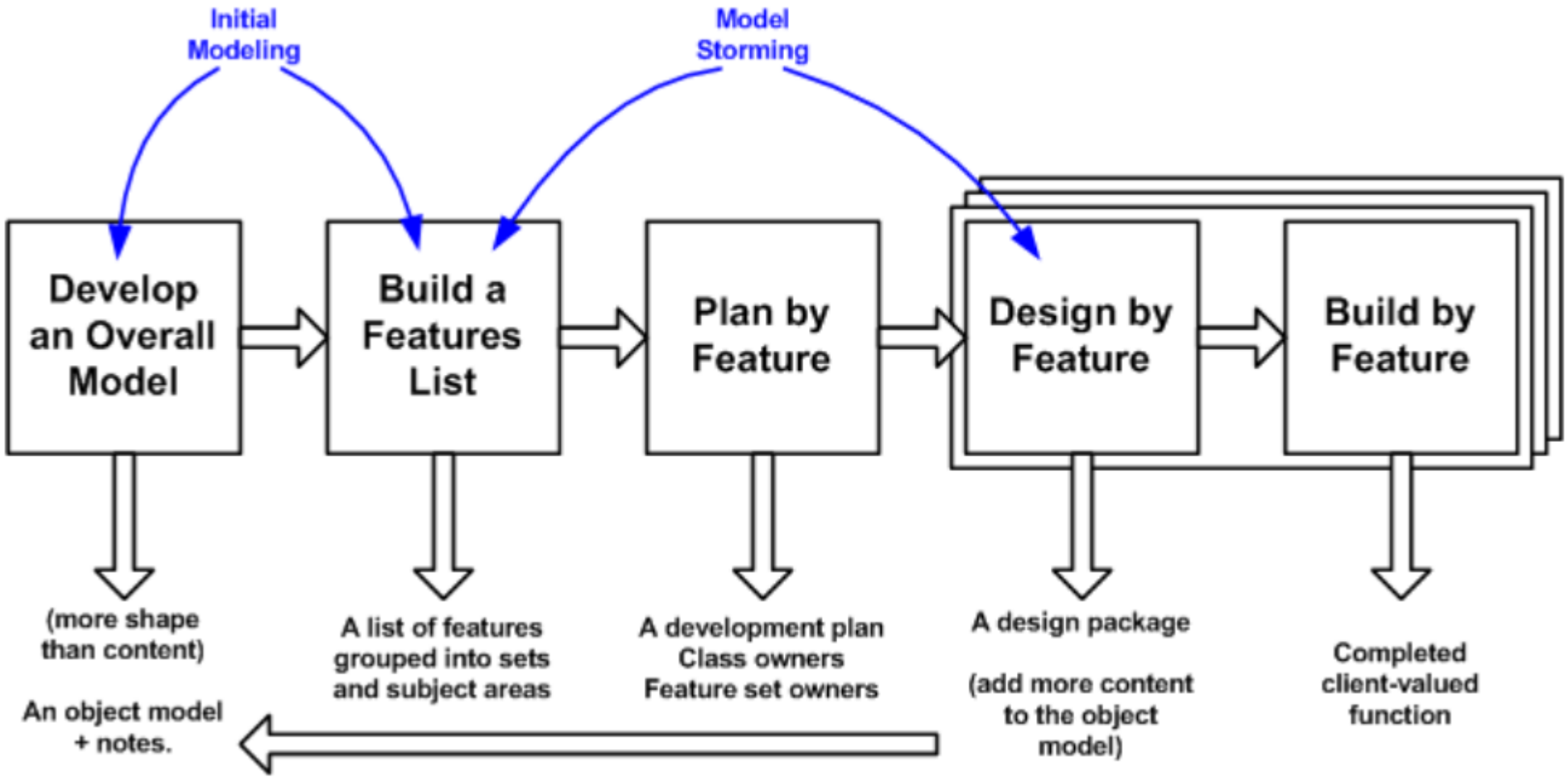
Conclusion

Feature-Driven Development is a practical Agile approach suited for long-term, complex projects. It is a suitable choice for development teams seeking a simple but structured Agile method that is scalable and delivers predictable results.

- **Best Practices:** FDD emphasizes domain object modeling, developing by feature, individual class ownership, feature teams, inspections, regular builds, and configuration management.
- **Key Roles:** FDD uses specialized roles including a Project Manager, Chief Architect, Development Manager, Chief Programmers, and Class Owners.
- **Benefits:** It is highly scalable for large teams, providing improved visibility through status reporting at all levels, resulting in reduced confusion and faster, high-quality delivery.
- **FDD vs. Scrum:** While Scrum focuses on time-boxed sprints, FDD focuses on the continuous, rapid development of functional features, often making it better suited for projects with a more defined, albeit large, initial scope.

FDD is ideal for projects that require a more structured approach than Scrum, offering a balance between agility and rigorous planning.

Figure 1. The FDD lifecycle.



Copyright 2002-2005 Scott W. Ambler
Original Copyright S. R. Palmer & J.M. Felsing

Agile Risk Management

Agile risk management is a proactive, iterative process integrated into daily and sprint-level activities to identify, assess, and mitigate threats to project scope, schedule, and quality. By emphasizing continuous communication, frequent client feedback, and small, incremental deliveries, teams can quickly address risks, reducing the likelihood of failure.

Risk Defined

"Anything that could prevent or marginalize project success – if it comes to fruition."

*Related, an **Issue** is when a **Risk** materializes.*

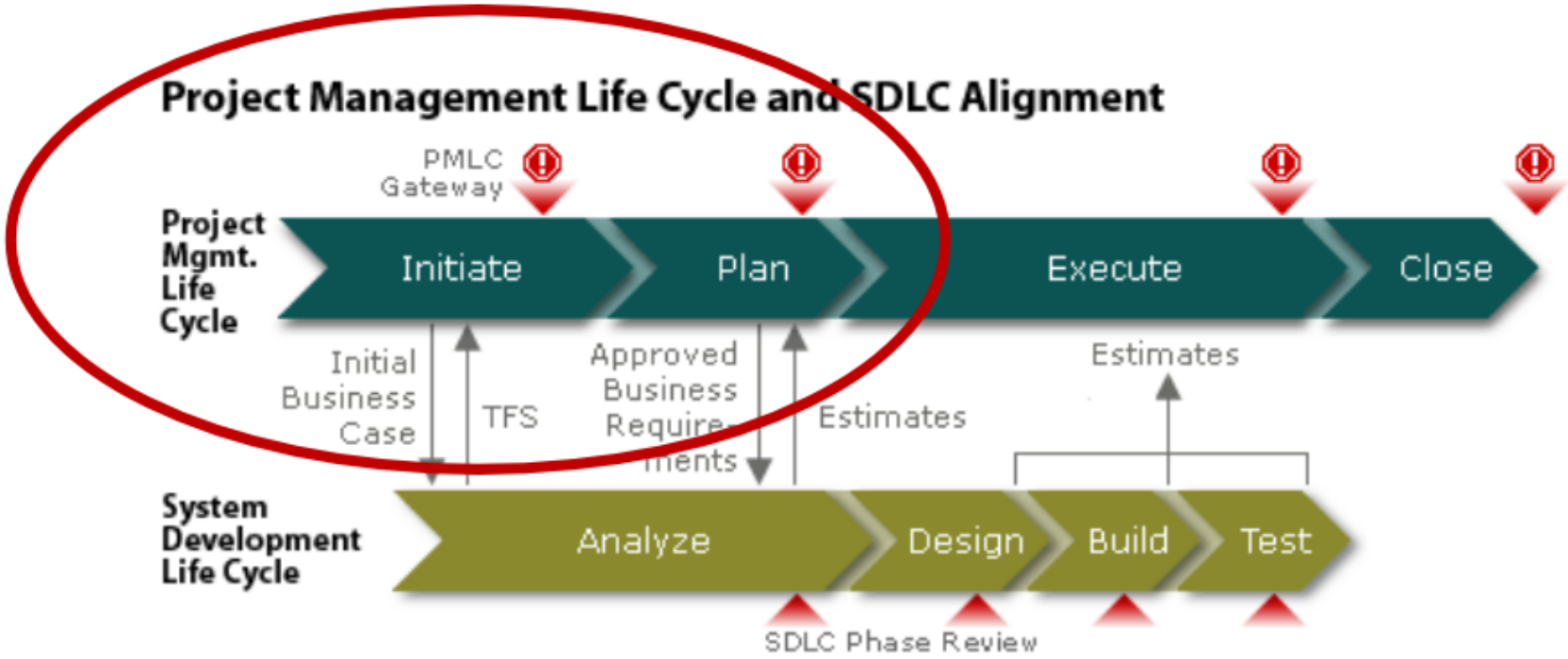
Frequent Risks

- The stakeholder requirements could be in conflict with each other.
- The estimate is not based on historical throughput.
- Team members are allocated to multiple projects.
- The project was approved without team buy-in.
- A 3rd party may not deliver their part.
  - Example - Hardware – bad weather in Asia could delay equipment

Traditional Risk Management Steps

1. Identify
2. Quantify Impact
3. Quantify Probability
4. Create contingencies for high impact - high probability risks
5. Manage highest scoring risks

When do we identify risks?



In our PMLC - usually once to ***Initiate*** the project, then revisit and update the risks after ***Planning***.

We can call this **BURP** – Big Upfront Risk Planning.

What Do We Identify?

Project Name:	Online Auction System
Project Number:	22880
Program Name:	0
Sponsor:	Jim Traylor
Business Owner:	Anne Archy
Project Manager:	Greg Smith

Risk Assessment

Risk ID Number	Risk Categories	Risk Description	Severity of Impact (1-5)	Likelihood of Occurring (in %)	Risk Rating
B5	Project Execution	Insufficient resources to successfully complete the project.	5	0.75	3.75

1. The potential risk
2. The impact if the risk happens
3. The likelihood the risk will occur
4. Ultimately a Risk Rating

We Usually Have Risk Categories

RISK CATEGORIES		
RISK CATEGORY	**DEFINITION**	**SAMPLE QUESTIONS**
Business Continuity	Includes risk associated with the duration, or impact, of an interruption of critical business processes and their associated people, vendors, systems, technology,	• Will the introduction of a new product or service cause an interruption to existing business processes? • Is there a Business Continuity Plan? • Has the Business Continuity Plan been updated to reflect changes?
Compliance	Includes risks introduced to the company either during the project, or as a result of the project, associated with failure to meet regulatory requirements.	• Is this a new process, product or business model that the Company has not had significant experience in implementing? • Has this project, product or process resulted in a customer impact resolution in the past? • Is this project in response to a new statute, regulation or comment from a regulator?
E-Commerce Risk	Risks associated with Internet interfaces	• Is web site privacy adequately protected?
		• Is the website and session security appropriately handled?
		• Is the access to data via web site adequately protected?
		• Is there protection in place to prevent hackers, denial of service attacks, website defacement, etc.?
		• Is the web site privacy adequately protected?
Financial	Includes operational risks associated with theft or misuse of Company or customer assets or information,	• Is the forecast of the financial performance of the project adequate? • Describe the assumptions used in this forecast and the risks to achieving them. • Are financial tracking requirements clearly documented? Are there changes to accounting practices?
Fraud/Theft	Includes risks introduced to the company either during the project, or as a result of the project, associated with theft or misuse of Company or customer assets or information,	

…and we have Risk Response categories

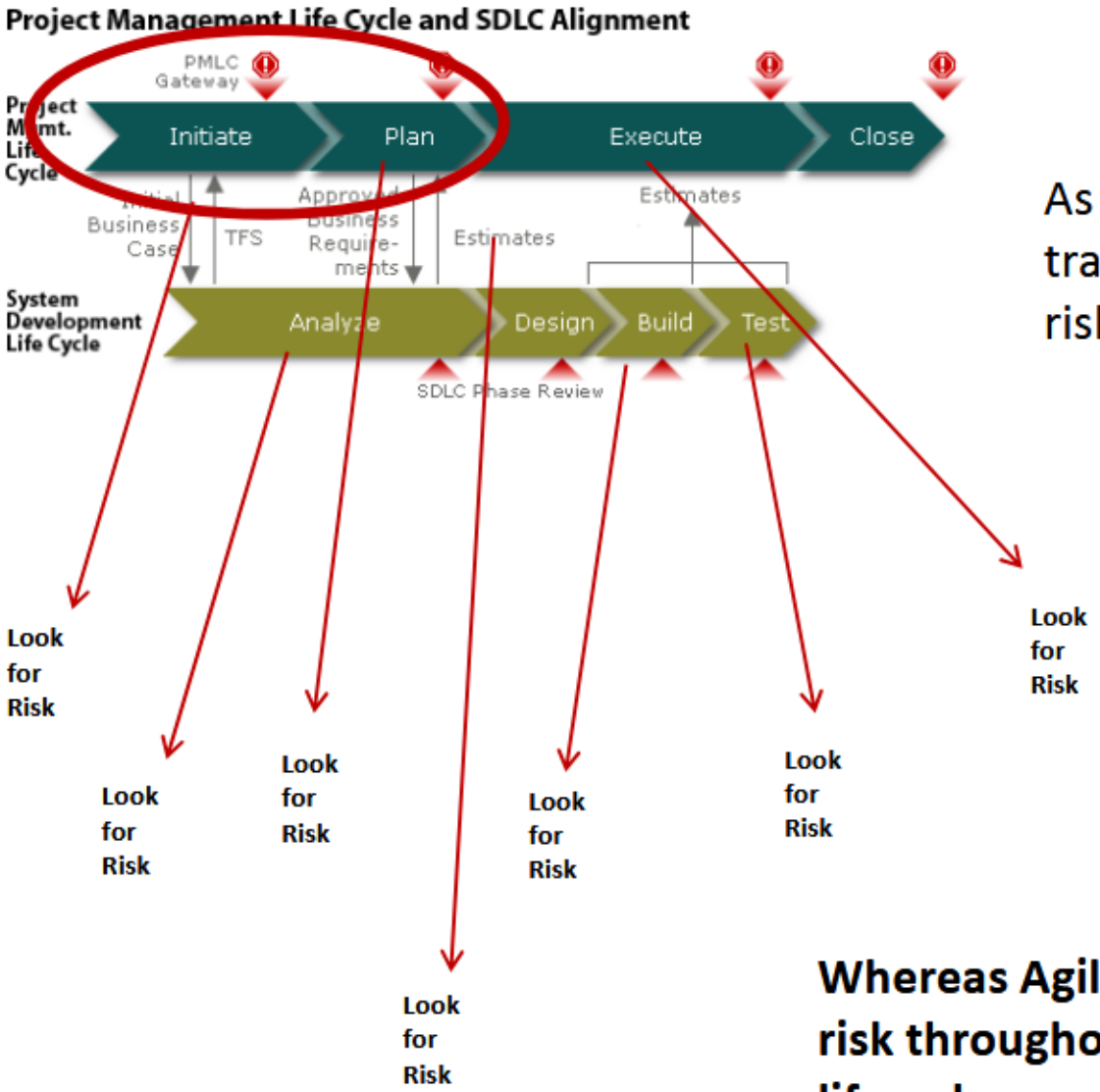
RISK RESPONSE APPROACHES	
Risk Avoidance	Eliminating the threat of a risk by eliminating the cause.
Mitigation (Controlling)	Reducing the consequences of a risk by reducing its severity of impact or likelihood of occurring.
Acceptance	Accepting the risk if it occurs.
Share or Transfer (Allocation)	Assigning the risk to another party by purchasing insurance or subcontracting.

When Done We Have a Scored List.

Risk ID Number	Risk Categories	Risk Description	Severity of Impact (1-5)	Likelihood of Occurring (in %)	Risk Rating	Contingency Plan Required (Yes/No)	Risk Approach	Contingency Plan Summary	Risk Owner
BUSINESS RISKS									
B1	Business Continuity	Incomplete migration of materials from vendor sites.	1	0.25	0.25	No	Acceptance	(none)	
B2	Compliance	Regulatory noncompliances related to inconsistent policies and procedures.	3	0.75	2.25	Yes	Exploit	Include as a KPI in business case – repeat survey question for Benefits Realization stage.	Jan Borgelt
B3	Human Resources	Publishers may not be competent in use of ecomm platform and tools.	3	0.5	1.5	No	Mitigation	Training & follow-up	
B4	Legal	Liability and/or reduced benefits arising from errors in termination of existing [illegible] contracts.	3	0.25	0.75	No	Risk Avoidance	(none)	
B5	**Project Execution**	**Insufficient resources to successfully complete the project.**	**5**	**0.75**	**3.75**	**Yes**	**Risk Avoidance**	**On Technology side, resources are involved with Core platform. Gabby starts another project on 9/24. Publishing department resources may not be available. We may draft Business representatives to continue working on**	**Greg: Technology; Larry: Business and overall budget**
B6	Project Execution	Anticipated project benefits could be reduced due to slow development, implementation, and migration. [immediate benefits]	3	0.75	2.25	Yes	Mitigation	Increase involvement of Business Owner and Sponsor.	Ann Archy
B7	Vendor Management/Outsourcin	A vendor may persuade a business manager not to end their relationship.	3	0.25	0.75	No	Acceptance	(none)	
B8	Human Resources	Negative employee perception of project related to vendor management and contract termination issues.	1	0.5	0.5	No	Acceptance	(none)	
B9	Vendor Management/Outsourcin	Reduced savings due to vendor-related issues.	3	0.25	0.75	No	Acceptance	(none)	
B10	Other	Employee perception of project failure related to unclear communications [illegible]	3	0.75	2.25	Yes	Mitigation	Communication: Address through communications plan	Abby

We create Contingency Plans for Risks with a High Rating

How is Agile different?



As we mentioned, traditional planning does risk management upfront.

Whereas Agile looks for risk throughout the lifecycle.

Agile Principles Address Risk

Transparency – Expose everything we are doing so we can see risks early

Collaborative planning – Harness the knowledge of the entire team and see more risks

Customer involvement – Mitigate customer risk by involving them throughout the lifecycle

Project *Envisioning* Practices

Envisioning the product with the customer

- The team and customer are synchronized on the need
- Less risk of delivering the wrong product

Quantifying the value with the customer

- Less risk of the team not supporting the project

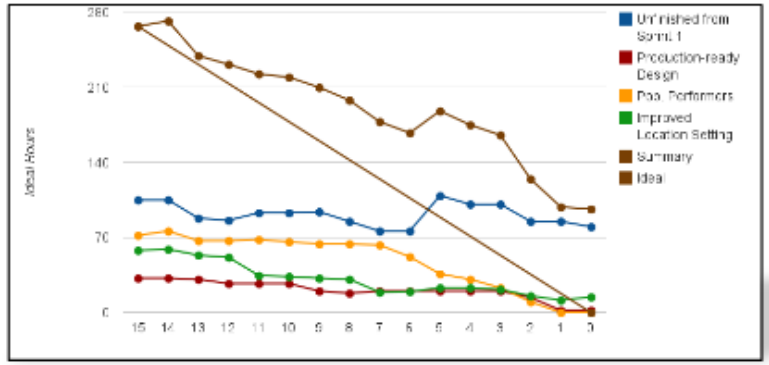
PROJECT ELEVATOR STATEMENT

For:	**Internet Retailers**
Who:	**Would like to sell their items locally within an auction framework**
The:	**Acme Auctionator**
Is a:	**Local Online auction system**
That:	**Allows the selling of goods without a commission**
And Unlike:	**Craigslist**
Our:	**Product allows the seller to put an item up for bid, as opposed to selling at a fixed price**

Project *Planning* Practices

Estimation based on history

- Risk of estimate inaccuracy reduced since constants are involved in estimation



Work reviewed at the feature level for more detailed risk evaluation:

- Less chance of missing a risk since features are examined separately for technical risk

Story ID: 2

Story Name: Ability to view seller information

Requirements Uncertainty: (H,M,L): Low

Technical Uncertainty: (H,M,L):: Low

Usage Frequency: (Daily, Weekly, Monthly, Other): Daily

Requirements Uncertainty: (H,M,L): Low

Technical Uncertainty: (H,M,L): Low

Dependencies w/other stories: Ability to post seller feedback

Acceptance: Bidder can see seller information summarized on items seller has for bid.

Development/Implementation Practices

Daily Standup Meetings

- Agile teams meet every 24 hours
- Which mean risks are exposed every 24 hours

Product Demos Every 2 to 4 weeks

- Constant exposure to the customer
- Minimizes the risk of building to the spec but not to the need

Project Tracking Risk Practices

Don’t Manage Based on % of Plan Complete

- Percentages are misleading
- There is a risk that 1% takes as long as 99%

	% Com	Task Name	Duration
	75%	Save as draft	10 days
	50%	Submit to legal	10 days
	99%	Discard Changes for iteration 1	10 days
	57%	**Backoffice Integration**	**10 days**
	47%	Fetch Data from CRM	10 days
✓	100%	Fetch Data from DWH	10 days
	32%	Fetch data from AD	10 days
	50%	Integration of web service - phase 1	10 days
	60%	**ID Numbers**	**10 days**
	20%	Create Main Contract ID	10 days
✓	100%	Create Sub Contract ID	10 days
	58%	**Associated Documents**	**10 days**
	45%	Upload - before submit to lega	10 days
	37%	Upload to library when submit to legal	10 days
	92%	Display associated docs in term sheet	10 days
	41%	**Look and feel**	**3 days**
	41%	Casey Design provides first pass on UI	3 days
	37%	**Versioning**	**24 days**
	37%	Design approved	10 days
	37%	Versioning in place	10 days

New Tasks : Auto Scheduled

Project Tracking Risk Practices

STORIES	Functional Requirements	Code Written	Unit Tested	System Integration	Functional Testing	Customer Approval	Load Test	DR Code Release	Production Code Release
Iteration 1									
Ability to register on the site	✓	✓	✓	✓	✓	✓	N/A		
Ability to place an item up for bid	✓	✓	✓	✓					
Ability to bid on an item	✓	✓	✓	✓	✓				
Auction Engine Logic									

Instead manage by binary attributes

- Complete or not complete
- Less risk of overrun on construction tasks

Key Aspects of Agile Risk Management

- **Iterative Process:** Risk management is not a one-time, up-front activity but a recursive cycle (identify, analyze, plan, implement, monitor) that occurs during every sprint.
- **Risk Identification & Prioritization:** Risks are identified through team meetings (daily stand-ups), retrospectives, and backlog refinement. They are prioritized using a formula of probability times impact.
- **Key Techniques:**
  - **Sprint Planning & Retrospectives:** Regular reviews allow teams to identify new risks and adjust plans.
  - **Backlog Refinement:** Helps prevent risks associated with misinterpreting requirements.
  - **Continuous Delivery:** Delivering small, working components often helps to identify issues early.
  - **Risk Register:** A document maintained to monitor known risks throughout the project.

- **Common Agile Risks:**
  - **Scope Creep:** Constantly changing requirements.
  - **Technical Debt:** Poorly maintained code causing future issues.
  - **Resource Constraints:** Lack of team availability or expertise.
  - **Unclear Requirements:** Misalignment with stakeholder expectations.
- **Benefits:** Enhanced flexibility, faster time-to-market, higher quality software, and increased customer satisfaction. PTC +7

Agile vs. Traditional Risk Management

Unlike traditional approaches that focus heavily on up-front planning, Agile focuses on managing risk through practical, ongoing actions. It is designed to be flexible, allowing for adjustments as project circumstances change. Agile Alliance +1

Implementing Best Practices for Quality Assurance in Agile-

When it comes to software development, quality is everything. Quality Assurance (QA) is a conventional method that guarantees an effective product and service. A strong QA unit explores the elements to design, produce, and develop excellent products thereby improving consumer trust, company reliability, and the capacity to succeed in an aggressive environment.

The Agile QA process starts at the origin of the software development life cycle. From the first design conference, in the development stage, to the final testing and setting of the application. This process is repeated in two-week sprints until the project is delivered.

What Is An Agile QA Process?

Most businesses have started the shift from the classical waterfall development methodology to an Agile method. Agile testing adds QA into the project as soon as feasible to anticipate problems, write and perform test cases, and reveal any gaps in time. With the project divided into iterative steps, QA engineers are ready to add focus to the development process and give rapid, continuous feedback. Sprints help the customer by delivering working software at the beginning rather than later, expecting innovation, providing more reliable assessments in less time, and providing for course changes instead of completely wrecking the project. The QA organization can incorporate models learned from past projects to develop the method for coming projects.

6 Best Practices for an Agile QA Process-

1. Risk Analysis

An essential focus of any QA method is risk analysis. Risk analysis is interpreted as the means of classifying and evaluating potential risks and their result. The process supports organizations to avoid and decrease chances.

It’s rare for an application to be 100% error-free, but a committed QA unit should strive to eliminate or restrict the most problematic errors. Knowing all the potential outcomes of a project enables your team to discover precautionary measures that decrease the likelihood of occurrence.

2. Test First also Test Often

The Agile model strives to include QA at each step of the project’s life cycle to know the effects as early as practicable. Within each sprint, QA technicians test and retest the product with every new feature added. This enables them to confirm that the new features were performed as required and to find any problems that may have been introduced. Testing first and often leads to the saving of time and resources.

3. White-box Vs Black-box

Black-box testing implies no knowledge of how a method does what it does. It merely has a knowledge of what it should do from the user’s view. White-box testing allows the QA engineer to generate a more profound knowledge of the system’s internals. Armed with this information, the QA engineer can start testing much quicker. Within an Agile QA process, the search engineers want this new level of practice understanding to verify features as soon as they are developed.

White-box testing enables QA units to predict potential failure conditions and produce better test situations. Knowing how the system works to ensure they have examined all possible input situations. It can also improve to the identification of potential security difficulties. Maybe most notable: white-box testing supports close collaboration between development and QA.

4. Automate If Feasible

Automation can help to maximize the effectiveness of your QA team better. Since regression examination can use a large portion of the QA team's time, self-regulation provides a way to secure previous deliverables and continue to work. At the same time, QA managers and engineers concentrate on testing recently delivered features. Being able to reproduce tests reliably will save up resources for exploratory testing. Automation will give your development team the courage to make adjustments to the system with the knowledge that any problems will be classified quickly, and can be fixed before delivery to the QA team.

All that being stated, it is essential to be careful of over-automating. Your team should prioritize test cases and later decide which of them should be automated. Conditions in which the data might vary or where a situation isn't consistently reproducible may not benefit from automation because the issues can cause false breakdowns.

Implementing automation costs more upfront, but protects money in the long run by improving efficiency between development and QA teams.

5. Know your Audience

Knowing your target audience will help to develop the QA process that is more relevant. Tailoring the expansion and QA process around your user's requirements will allow your team to create value-driving applications. When you are intimate with who will be practicing the real end-product, you can better prioritize the QA method to save time and money.

6. Teamwork Makes the Dream Work

Behind each high-quality product, there is a unit of professionals that regularly work to keep the high measure of quality upheld by the organization. Although each team running on the project must take accountability for assuring excellence, the primary responsibility for quality rests with the QA team. The roles and responsibilities of the QA team revolve around what the customer wants the system to do and can determine the client's comfort with the order.

Managing the Agile QA process, engineers are the super-sleuths who root out obstacles and support the team to deliver high-quality results and secure client trust, company reliability, and reliable product performance.

Agile Tools-

Agile tools for software development facilitate iterative planning, team collaboration, and progress tracking, with top choices including Jira, Trello, Asana, and ClickUp. These tools support Kanban and Scrum frameworks through backlog management, sprint planning, and real-time reporting. Essential categories include project management, version control (e.g., GitHub), and communication platforms.

Top Agile Software Development Tools

Jira: The market leader for Scrum and Kanban, offering in-depth reporting (burndown charts, velocity) and issue tracking.
Trello: A user-friendly, Kanban-based tool ideal for simple task management and visualization.
ClickUp: Known for high customization, offering all-in-one productivity with built-in docs and goal tracking.
Asana: Excellent for team collaboration, task tracking, and mapping out workflows.
Azure DevOps: A comprehensive tool for planning, tracking, and CI/CD pipelines.
Confluence: A collaboration tool for documentation that integrates directly with Jira.
Slack: Widely used for real-time team communication and integrating with other tools.

Common Agile Frameworks Supported

Scrum: Focused on iterative, time-boxed sprints.
Kanban: Focused on visualizing work and optimizing flow.
Extreme Programming (XP): Emphasizes technical practices like Test-Driven Development (TDD) and pair programming.

What is Agile Testing

Agile Testing is a Type of Software Testing that follows the principles of agile software development to test the software application. All members of the project team, along with the special experts and testers, are involved in agile testing. Agile testing is not a separate phase, and it is carried out with all the development phases, which are requirements, design, coding, and test case generation.

Agile testing is a flexible and dynamic process that runs continuously throughout each iteration of the **Software Development Life Cycle (SDLC)**. The main focus for Agile testers is customer satisfaction, verifying that the product meets the needs and expectations of the users.

- Agile testing is an informal process that is specified as a dynamic type of testing.
- It is performed regularly throughout every iteration of the Software Development Lifecycle (SDLC).
- Customer satisfaction is the primary concern for agile test engineers at some stage in the agile testing process.

Agile Testing Principles

Agile testing combines traditional testing with development to provide continuous feedback, faster fixes, and better alignment with customer needs. The main principles of Agile testing focus on:

1. **Shortening feedback iteration:** In Agile Testing, the testing team gets to know the product development and its quality for each and every iteration. Thus continuous feedback minimizes the feedback response time, and the fixing cost is also reduced.
2. **Testing is performed alongside** Agile testing is not a different phase. It is performed alongside the development phase. It ensures that the features implemented during that iteration are actually done. Testing is not kept pending for a later phase.
3. **Involvement of all members:** Agile testing involves each and every member of the development team and the testing team. It includes various developers and experts.

4. **Documentation is weightless:** In place of global test documentation, agile testers use reusable checklists to suggest tests and focus on the essence of the test rather than the incidental details. Lightweight documentation tools are used.
5. **Clean code:** The defects that are detected are fixed within the same iteration. This ensures clean code at any stage of development.
6. **Constant response:** Agile testing helps to deliver responses or feedback on an ongoing basis. Thus, the product can meet the business needs.
7. **Customer satisfaction:** In agile testing, customers are exposed to the product throughout the development process. Throughout the development process, the customer can modify the requirements, and update the requirements and the tests can also be changed as per the changed requirements.
8. **Test-driven:** In agile testing, the testing needs to be conducted alongside the development process to shorten the development time. But testing is implemented after the implementation or when the software is developed in the traditional process.

Agile Testing Methodologies

Agile testing methodologies focus on flexibility, collaboration, and continuous improvement. Here are some key Agile testing methods explained in simple terms:

1. Test-Driven Development (TDD): In TDD, tests are written before writing the actual code. This approach involves three steps: writing a unit test, coding to pass the test, and then refactoring the code. It verify that the code is always tested and improved in small, manageable steps.

2. Behavior Driven Development (BDD): BDD is all about understanding and testing how users interact with the application. It focuses on creating features based on user behavior and encourages collaboration between developers, testers, and customers to ensure the software meets user expectations.

3. Exploratory Testing: Here, testers are free to explore the software as they see fit, without following predefined test scripts. This method helps uncover unknown risks and bugs by allowing testers to test the software in creative and flexible ways.

4. Acceptance Test-Driven Development (ATDD): ATDD involves the customer, developers, and testers working together to define the requirements and potential challenges before coding begins. This collaborative effort reduces the chance of errors and helps build software that meets customer needs from the start.

5. Extreme Programming (XP): XP is focused on delivering high-quality software that meets customer needs. It involves practices that emphasize customer involvement, simplicity, and frequent releases to ensure the final product aligns with customer expectations.

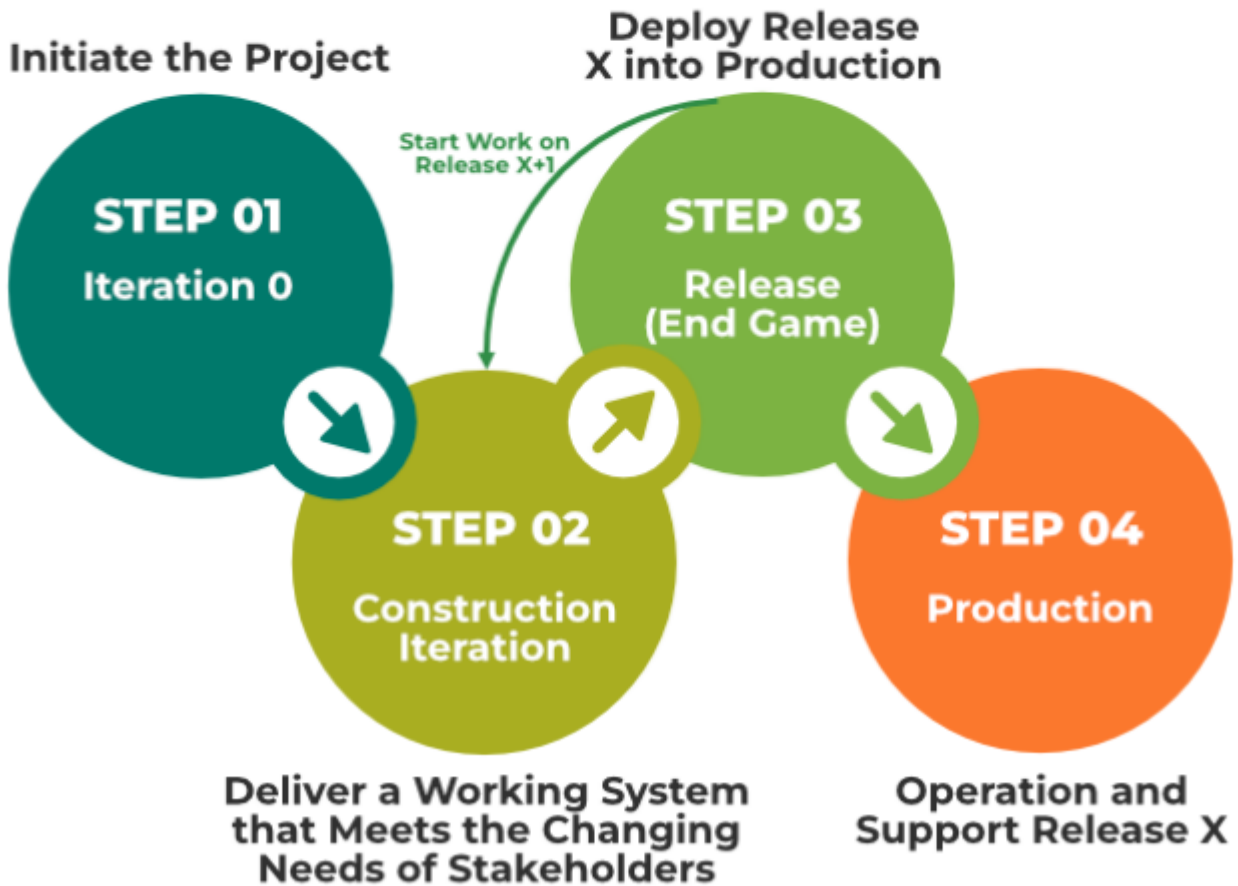
6. Session-Based Testing: This method involves structured, time-limited testing sessions. Testers focus on different aspects of the software within a set time frame (usually 45 to 90 minutes), during which they document their findings in a charter document. This ensures focused and efficient testing.

7. Dynamic Software Development Method (DSDM): DSDM is an Agile framework for delivering projects. It provides a set of principles for developers, users, and testers to collaborate and build systems that meet the needs of the business.

8. Crystal Methodologies: The Crystal methodology focuses on the people involved in a project rather than on processes or tools. It adapts to the size and criticality of the project, emphasizing communication, flexibility, and simplicity to deliver results effectively.

Agile Testing Strategies

Agile testing is all about flexibility and verifying the quality at every stage of the development process. It happens in distinct phases, with each one playing an important role in helping teams adapt and deliver high-quality software.



1. Iteration 0

It is the first stage of the testing process and the initial setup is performed in this stage. The testing environment is set in this iteration. This stage involves executing the preliminary setup tasks such as finding people for testing, preparing the usability testing lab, preparing resources, etc. The business case for the project, boundary situations, and project scope are verified.

- Important requirements and use cases are summarized.
- Initial project and cost valuation are planned.
- Risks are identified.
- Outline one or more candidate designs for the project.

2. Construction Iteration

It is the second phase of the testing process. It is the major phase of the testing and most of the work is performed in this phase. It is a set of iterations to build an increment of the solution. This process is divided into two types of testing:

- **Confirmatory testing:** This type of testing concentrates on verifying that the system meets the stakeholder's requirements as described to the team to date and is performed by the team. It is further divided into 2 types of testing:
    - **Agile acceptance testing:** It is the combination of acceptance testing and functional testing. It can be executed by the development team and the stakeholders.
    - **Developer testing:** It is the combination of unit testing and integration testing and verifies both the application code and database schema.
- **Investigative testing:** Investigative testing detects the problems that are skipped or ignored during confirmatory testing. In this type of testing, the tester determines the potential problems in the form of defect stories. It focuses on issues like integration testing, load testing, security testing, and stress testing.

3. Release End Game

This phase is also known as the transition phase. This phase includes the full system testing and the acceptance testing. To finish the testing stage, the product is tested more relentlessly while it is in construction iterations. In this phase, testers work on the defect stories. This phase involves activities like:

- Training end-users.
- Support people and operational people.
- Marketing of the product release.
- Back-up and restoration.
- Finalization of the system and user documentation.

4. Production

It is the last phase of agile testing. The product is finalized in this stage after the removal of all defects and issues raised.

Agile Testing Quadrants

Agile testing is divided into four quadrants, each focusing on a specific aspect of the testing process.

1. Quadrant 1 (Automated)

The first agile quadrat focuses on the internal quality of code which contains the test cases and test components that are executed by the test engineers. All test cases are technology-driven and used for automation testing.

All through the agile first quadrant of testing, the following testing can be executed:

- **Unit testing**: This involves testing individual parts or functions of your code to make sure they work correctly on their own.
- **Component testing**: This is about testing bigger sections or modules of the code. It's a step up from unit testing, where you're validating how different pieces of code work together as a whole.

2. Quadrant 2 (Manual and Automated)

The second agile quadrant focuses on the customer requirements that are provided to the testing team before and throughout the testing process. The test cases in this quadrant are business-driven and are used for manual and automated functional testing.

The following testing will be executed in this quadrant:

- **Pair testing**: This is when two testers work together to find defects in the software. By collaborating, testers can share ideas, catch issues more quickly, and improve the overall testing process.
- **Testing scenarios and workflow:** This involves validating that the application works as expected according to business requirements.
- **User Stories and Prototypes:** Testing user stories and prototypes helps verify that the software meets user expectations.

3. Quadrant 3 (Manual)

The third agile quadrant provides feedback to the first and the second quadrant. This quadrant involves executing many iterations of testing, these reviews and responses are then used to strengthen the code. The test cases in this quadrant are developed to implement automation testing.

The testing that can be carried out in this quadrant are:

- **Usability Testing:** This is about checking how easy and user-friendly the application is.
- **Collaborative Testing:** In this activity, testers work closely with other team members and stakeholders to identify problems and improve the product.
- **User Acceptance Testing (UAT):** This step ensures that the product meets the needs and expectations of the end users.
- **Pair Testing with Customers:** Testing alongside customers helps gather real-time feedback about the user experience.

4. Quadrant 4 (Tools)

The fourth agile quadrant focuses on the non-functional requirements of the product like performance, security, stability, etc. Various types of testing are performed in this quadrant to deliver non-functional qualities and the expected value.

The testing activities that can be performed in this quadrant are:

- Security testing.
- Scalability testing.
- Infrastructure testing
- Data migration testing.
- Non-functional testing such as stress testing, load testing, performance testing, etc.

Types of Security Testing

Security testing is important to check and sure that applications and systems are protected from various threats. There are several types of security testing, each targeting specific vulnerabilities and aspects of security.

Security testing includes various methods, each targeting specific vulnerabilities:

Scalability Testing is a type of non-functional testing in which the performance of a software application, system, network or process is tested in terms of its capability to scale up or scale down the number of user request load or other such performance attributes. It can be carried out at a hardware, software or database level. Scalability Testing is defined as the ability of a network, system, application, product or a process to perform the function correctly when changes are made in the size or volume of the system to meet a growing need. It ensures that a software product can manage the scheduled increase in user traffic, data volume, transaction counts frequency and many other things. It tests the system, processes or database's ability to meet a growing need.

What is Stress Testing?

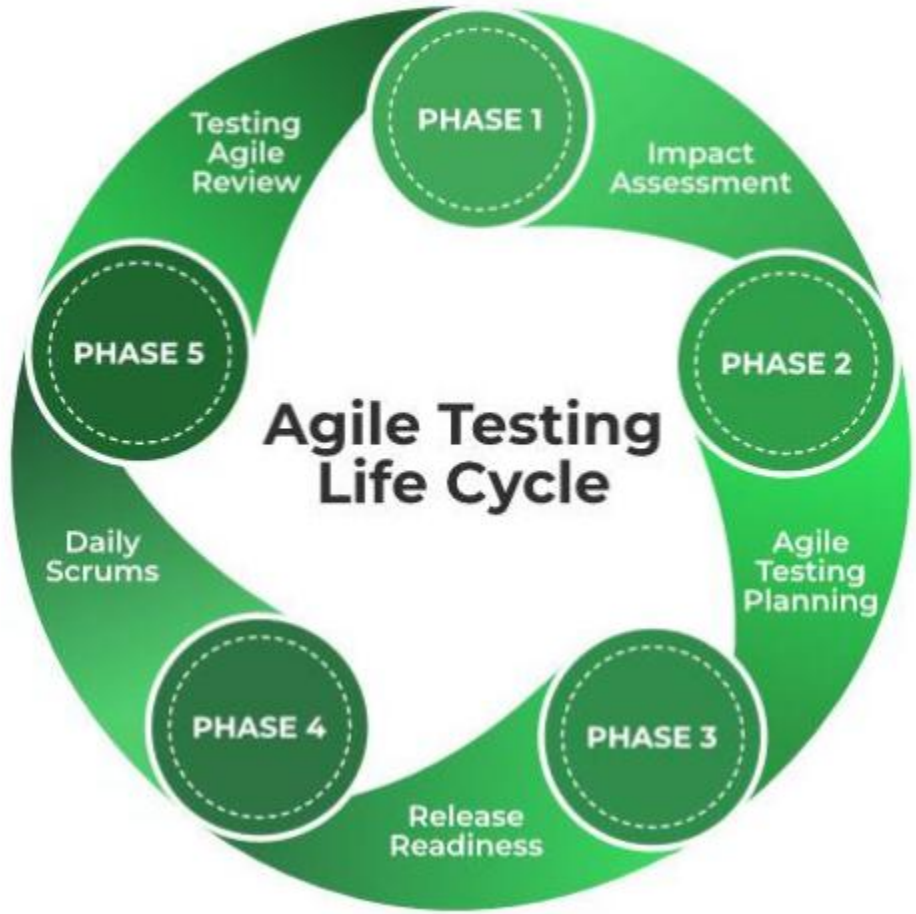
Stress testing is defined as types of software testing that verifies the stability and reliability of the system. This test particularly determines the system's robustness and error handling under the burden of some load conditions. It tests beyond the normal operating point and analyses how the system works under extreme conditions. Stress testing is performed to ensure that the system does not crash under crunch situations. Stress testing is also known as **Endurance Testing** or **Torture Testing** .

Load testing is a type of Performance Testing that determines the performance of a system, software product, or software application under real-life-based load conditions. Load testing determines the behavior of the application when multiple users use it at the same time. It is the response of the system measured under varying load conditions.

1. The load testing is carried out for normal and extreme load conditions.
2. Load testing is a type of performance testing that simulates a real-world load on a system or application to see how it performs under stress.
3. The goal of load testing is to identify bottlenecks and determine the maximum number of users or transactions the system can handle.
4. It is an important aspect of software testing as it helps ensure that the system can handle the expected usage levels and identify any potential issues before the system is deployed to production.

Agile Testing Life Cycle

The agile testing life cycle has 5 different phases:



1. **Impact Assessment:** This is the first phase of the agile testing life cycle also known as the feedback phase where the inputs and responses are collected from the users and stakeholders. This phase supports the test engineers to se the objective for the next phase in the cycle.
2. **Agile Testing Planning:** In this phase, the developers, customers, test engineers, and stakeholders team up to plan the testing process schedules, regular meetings, and deliverables.
3. **Release Readiness:** This is the third phase in the agile testing lifecycle where the test engineers review the features which have been created entirely and test if the features are ready to go live or not and the features that need to be sent again to the previous development phase.
4. **Daily Scrums:** This phase involves the daily morning meetings to check on testing and determine the objectives for the day. The goals are set daily to enable test engineers to understand the status of testing.
5. **Test Agility Review:** This is the last phase of the agile testing lifecycle that includes weekly meetings with the stakeholders to evaluate and assess the progress against the goals.

Agile Test Plan

An agile test plan includes types of testing done in that iteration like test data requirements, test environments, and test results. In agile testing, a test plan is written and updated for every release. The test plan includes the following:

1. Test Scope.
2. Testing instruments.
3. Data and settings are to be used for the test.
4. Approaches and strategies used to test.
5. Skills required to test.
6. New functionalities are being tested.
7. Levels or Types of testing based on the complexity of the features.
8. Resourcing.
9. Deliverables and Milestones.
10. Infrastructure Consideration.
11. Load or Performance Testing.
12. Mitigation or Risks Plan.

Benefits of Agile Testing

Below are some of the benefits of agile testing:

- **Saves time:** Implementing agile testing helps to make cost estimates more transparent and thus helps to save time and money.
- **Reduces documentation:** It requires less documentation to execute agile testing.
- **Enhances software productivity:** Agile testing helps to reduce errors, improve product quality, and enhance software productivity.
- **Higher efficiency:** In agile software testing the work is divided into small parts thus developer can focus more easily and complete one part first and then move on to the next part. This approach helps to identify minor inconsistencies and higher efficiency.
- **Improve product quality:** In agile testing, regular feedback is obtained from the user and other stakeholders, which helps to enhance the software product quality.

Limitations of Agile Testing

Below are some of the limitations of agile software testing:

- **Project failure:** In agile testing, if one or more members leave the job then there are chances for the project failure.
- **Limited documentation:** In agile testing, there is no or less documentation which makes it difficult to predict the expected results as there are explicit conditions and requirements.
- **Introduce new bugs:** In agile software testing, bug fixes, modifications, and releases happen repeatedly which may sometimes result in the introduction of new bugs in the system.
- **Poor planning:** In agile testing, the team is not exactly aware of the end result from day one, so it becomes challenging to predict factors like cost, time, and resources required at the beginning of the project.
- **No finite end:** Agile testing requires minimal planning at the beginning so it becomes easy to get sidetracked while delivering the new product. There is no finite end and there is no clear vision of what the final product will look like.

Challenges During Agile Testing

Below are some of the challenges that are faced during agile testing:

- **Changing requirements:** Sometimes during product development changes in the requirements or the specifications occur but when they occur near the end of the sprint, the changes are moved to the next sprint and thus become the overhead for developers and testers.
- **Inadequate test coverage:** In agile testing, testers sometimes miss critical test cases because of the continuously changing requirements and continuous integration. This problem can be solved by keeping track of test coverage by analyzing the agile test metrics.
- **Tester's availability:** Sometimes the testers don't have adequate skills to perform API and Integration testing, which results in missing important test cases. One solution to this problem is to provide training for the testers so that they can carry out essential tests effectively.
- **Less Documentation:** In agile testing, there is less or no documentation which makes the task of the QA team more tedious.

- **Performance Bottlenecks:** Sometimes developer builds products without understanding the end-user requirements and following only the specification requirements, resulting in performance issues in the product. Using load testing tools performance bottlenecks can be identified and fixed.
- **Early detection of defects:** In agile testing, defects are detected at the production stage or at the testing stage, which makes it very difficult to fix them.
- **Skipping essential tests:** In agile testing, sometimes agile testers due to time constraints and the complexity of the test cases put some of the non-functional tests on hold. This may cause some bugs later that may be difficult to fix.

Risks During Agile Testing

- **Automated UI slow to execute:** Automated UI gives confidence in the testing but they are slow to execute and expensive to build.
- **Use a mix of testing types:** To achieve the expected quality of the product, a mixture of testing types and levels must be used.

- **Poor Automation test plan:** Sometimes automation tests plan is poorly organized and unplanned to save time which results in a test failure.
- **Lack of expertise:** Automated testing sometimes is not the only solution that should be used, it can sometimes lack the expertise to deliver effective solutions.
- **Unreliable tests:** Fixing failing tests and resolving issues of brittle tests should be the top priority to avoid false positives.

What is user acceptance testing (UAT)?

User Acceptance Testing (UAT) serves the purpose of ensuring that the software meets the **business requirements** and is ready for **deployment** by validating its functionality in a real-world environment. It allows **end-users** to test the software to ensure it meets their needs and operates as expected, helping to identify and fix any issues before the final release. UAT is crucial for **quality assurance** and **customer satisfaction**, as it ensures that the software is **user-friendly**, **reliable**, and meets all specified **criteria**.

Acceptance criteria attributes for UAT

- Completeness
- Accuracy
- User-friendliness
- Performance
- Reliability
- Security
- Scalability
- Compatibility

Who performs UAT?

User Acceptance Testing (UAT) is typically performed by **end-users** or **clients** who will ultimately use the software in their daily operations. These individuals represent the target audience for the software and are responsible for validating whether the software meets their requirements and expectations before it is deployed.

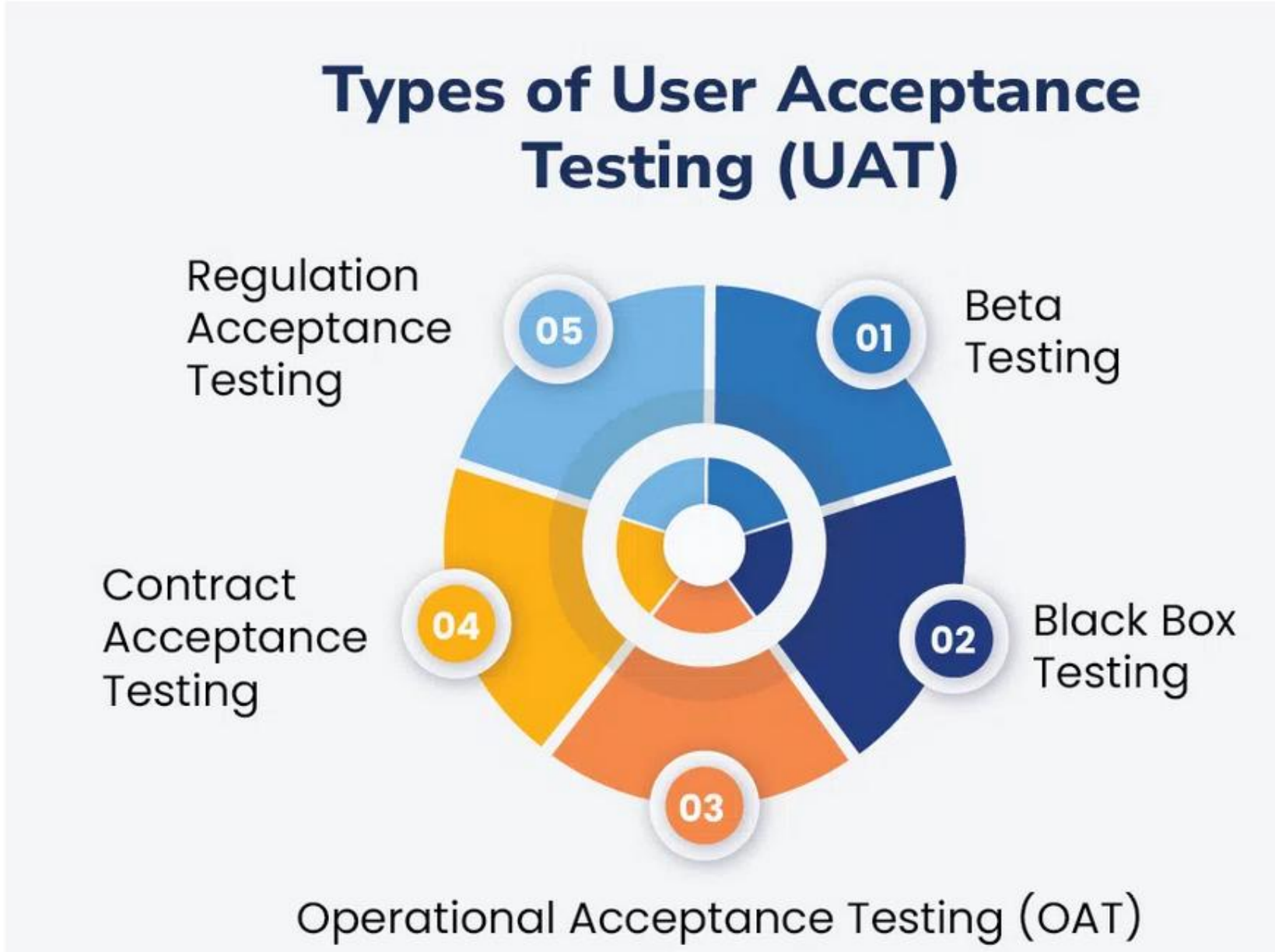
What is the purpose of UAT?

The purpose of User Acceptance Testing (UAT) is to identify bugs in software, systems, and networks that may cause problems for users. UAT ensures that software can handle real-world tasks and perform to development specifications. Users are allowed to interact with the software before its official release to see if any features were overlooked or if any bugs exist.

Basic guidelines of UAT: When doing UAT, it is important to test with users in different locations, not just on different devices. It is better to have one person in a separate location rather than testing with people who are already online together. Also, using email is often more reliable than social media, especially for private communication, like delivering services through apps like Signal.

Types of User Acceptance Testing

Below are the 5 types of user acceptance testing:



1. Beta User Acceptance Testing

- Beta UAT means that users who have completed one or more rounds of tests will be shown a popup stating if they are accepted for testing by the new version of Angular2 (a beta release).
- The application is tested in a natural environment.
- It reduces risks, and failures, and improves the quality of the product through customer feedback.

2. Black Box Testing

- In **black box testing**, end-users or testers evaluate specific functionalities of the software without knowing the internal workings or code structure.
- Testers focus on how well the software performs its intended tasks from a user perspective, checking inputs and outputs against expected outcomes.
- This type of testing ensures that the software meets user requirements without requiring knowledge of its underlying technical details.

3. Operational Acceptance Testing (OAT)

- Operational Acceptance Testing (OAT) is a software testing technique that evaluates a software application's operational readiness before release or production.
- The goal of operational acceptance testing is to ensure system and component compliance as well as the smooth operation of the system in its Standard Operating Environment (SOE).
- OAT Testing (Operational Acceptance Testing) is also known as Operational Readiness Testing (ORT) or Operational Testing.
- These test cases guarantee there are work processes set up to permit the product or framework to be utilized.
- This ought to incorporate work processes for reinforcement plans, client preparation, and different support cycles and security checks.

4. Contract Acceptance Testing

- Contract Acceptance Testing refers to the process of testing developed software against predefined and agreed-upon criteria and specifications.
- When the project team agrees on the contract, they define the relevant criteria and specifications for acceptance.
- Contract acceptance testing involves testing the software against specific criteria and specifications outlined in the project contract or agreement.
- This type of UAT ensures that the delivered software aligns with the agreed-upon terms and conditions between the client and the development team.

5. Regulation Acceptance Testing

- Regulation AT is generally called Compliance AT.
- This sort of affirmation testing is done to guarantee the thing dismisses no rules and rules that are set by the regulating associations of the particular country where the thing is being conveyed.
- Generally, things that are available from one side of the planet to the other should go through this testing type considering the way that different countries have different standards and rules set by discrete directing associations.

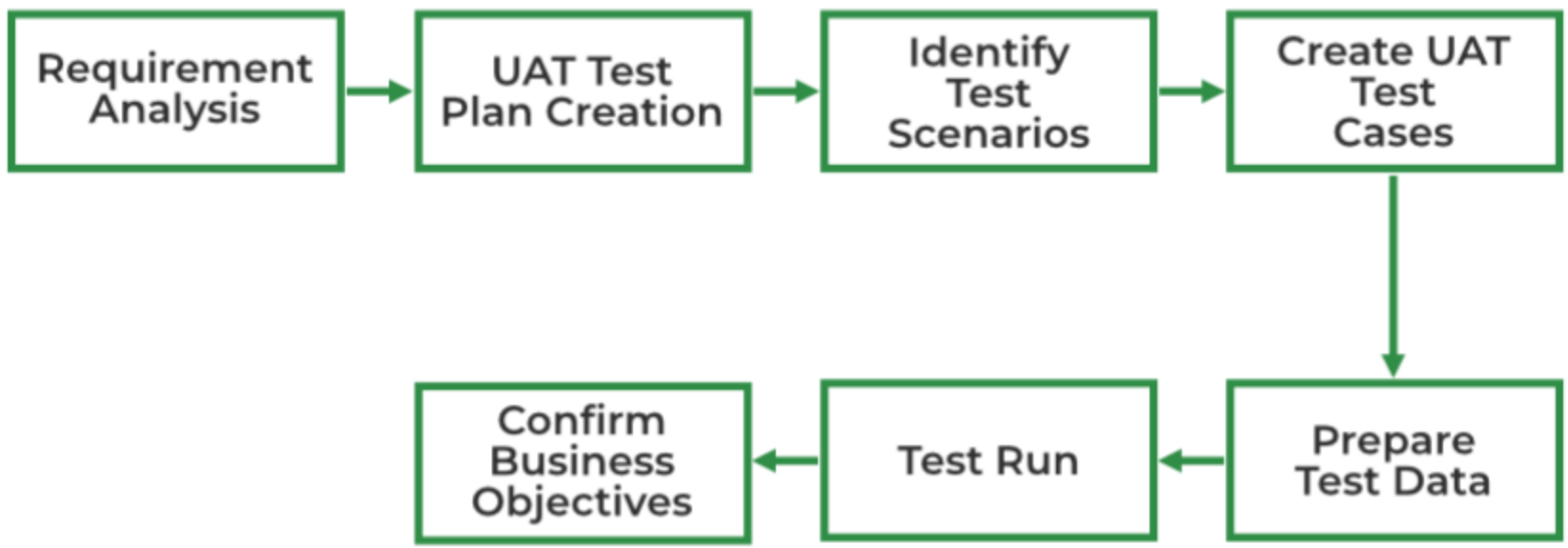
6. Alpha User Acceptance Testing

- Alpha UAT means that your user is tested before they get a hold of the product, so if you're testing users' usage patterns we recommend running an alpha test to ensure it can pass all acceptance tests before the beta gets deployed into production.
- It enables more rapid validation in early adopters/testers which allows fast adjustments as the software progresses through development with each release cycle toward feature maturity.
- It ensures that there is no opportunity for bugs or exploits once security updates become available based upon adoption levels achieved by products launched later during their life cycle such should be done at least six months after launch.

How to perform User Acceptance Testing (UAT)

Each type of UAT serves a specific purpose in validating the software's functionality, usability, and compliance before it is deployed for production use.

Here are the steps of Execute User Acceptance Tests



1. Requirements Analysis

This step involves analyses of business requirements. The following documents will be considered and studied thoroughly to identify and develop test scenarios:

- Business Use Cases.
- Business Requirements Document (BRD).
- System Requirements Specification (SRS).
- Process Flow Diagrams.

2. UAT Test Plan Creation

In this step, a test place is created that will help to outline the test strategy that will be used to verify and ensure that the software meets the expected business requirements. The test plan includes entry criteria, exit criteria, test scenarios, and a test case approach.

3. Identify Test Scenarios

This step involves identifying the test scenarios will respect to the business requirements and creating test cases listing the clear test steps. The test cases should cover the UAT test scenarios.

4. Create UAT Test Cases

Create UAT test cases in this step that cover most of the test scenarios. Business use cases are the input here to create test cases.

5. Prepare Test Data

It is considered a best practice to use live data for UAT testing, UAT testers should be familiar with database flow.

6. Test Run

This step involves executing the test cases and reporting the bugs if there are any. Re-test the software once the bugs are fixed. In this step, test management tools can be used for test case execution.

7. Confirm Business Objectives

In this step, the UAT testers need to sign off the mail after the UAT testing to ensure that the product is good to go for production. Deliverables here are Test Plan, UAT Test Scenarios, Test Cases, Results Log, and Defect Log.

Challenges of User Acceptance Testing (UAT)

Challenges of carrying out User Acceptance Testing include:

- **Misreporting Activities:** The use and misuse/misreporting activities by potential users can be extremely challenging to control. For example, this issue may arise when a company is not equipped with appropriate information systems.
- **Proper Example to demonstrate:** Provide an example project to test the validity and reliability, or at least performance, aspects - such as time complexity, resource usage per user, etc.
- **Proper Evaluation:** Evaluating how this information is handled by users after a successful acceptance Test needs to be done using common programming tools that provide adequate input data including HTML formatted examples with optional inputs for feedback before/after each iteration.
- **Usability:** The tester's job is critical in UAT since they must demonstrate the usability of your product by simulating real-life scenarios. They must also gather information on how your users interact with your product.

- **Proper Balance:** In addition to inviting users, IT professionals must balance user input and expectations with costs and constraints. For example, some companies limit the number of users per computer during their beta tests. This limits both costs and data collection.
- **Limitations of Actions Performed by User:** There are also limitations on what actions each user can perform within the program— for example, some programs have an expiration date so that companies do not waste valuable data on unappealing customers.

Tools for User Acceptance Testing

A few tools used for UAT are listed below:

1. Marker.io

Report visual bugs straightforwardly into your devices, without leaving your site or web application

- It lets users post messages, comments, and events to a "hub" hosted on Google Analytics, with an optional delay between updates which ensures only one message gets sent per second.
- This delays your data loss by eliminating any accidental user interactions that might interrupt their Web App flow.

2. FullStory

Enables clients to track and screen every client action. From snaps to page advances, everything is listed consequently.

- It allows you to visualize user acceptance and rejection through some graphs, similar in functionality to GraphPad but with a lot more flexibility.
- The data can be viewed either via interactive dashboards like Scrum or by drawing on individual parts of it that are then visualized along with actual user feedback using your favorite software It makes this kind of structured test much easier than one would typically think, perhaps even less frustrating.

3. Hotjar

Uncovers the internet-based conduct and voice of your clients. Hotjar provides you with the '10,000-foot view' of how to further develop your site's client experience and execution/transformation rates.

- This application runs a service that keeps track of an online database of people who have ever viewed your website.
- The following page summarizes what Hotjars offer and provides tips on creating websites using them.
- Also, It allows users to run tests from a command line and it does a great job at testing various features that may be added later on.

4. CrazyEgg

A web-based device that screens individual pages from your site, providing you with a breakdown of where various guests have clicked and on what part of the screen.

- The user will need to build a class with all needed methods and return values along its arguments so that this can be easily tested by other developers or clients/users using different APIs like Selenium Server test suites.
- It comes in two flavors - one which builds on top of Mocha Test Suite i.e. WebDriver, and the other has just built upon MuleTest's framework but adds some custom features such as implementation through Sockets, etc.

5. Qualaroo

Allows users to easily test their Web Apps.

- Qualaroo is a Python library that allows users to easily test their Web Apps.
- Common data structures can be created in Python which allows us to directly run our tests against different server configurations using QA tools like RSpec and TDDRunner.

6. Sentry

A web interface that allows users to write acceptance tests on their own

- It's simple but effective and has been accepted into several national standards bodies such as ISO 9001 and ANSI X9-TRIAMS.
- Sentry provides a web interface that allows users to write acceptance tests and upload them by selecting an option on their dashboard from the toolbar menu with various test cases selected during setup.

Exit Criteria for User Acceptance Testing

There are some Exit Criteria required to be met for User Acceptance Testing. They include:

- **Confidence:** A high level of confidence that the proposed user has enough knowledge, experience, and skill set to perform at least one task effectively.
- **Proper Execution:** Where tests show users can contribute fully to existing tasks successfully using their expertise. All three terms represent different levels with each being less than 50% when compared to full-time professionals in this area. When you use these two criteria as input your goal is to gain support from others who have achieved similar results through other research methods instead of focusing on just learning how important it was once they got there.

- **Lesser Defects:** After analyzing the test results, project managers should be able to draw some conclusions based on what they've found. For example, if there are more errors during testing than expected, this can be taken as a positive sign. It shows that the program is easy to learn and use which is a necessary condition for successful implementation. In addition, this means that their project objectives are understandable and easily implemented by end users. In other words, their business process works satisfactorily. If there are fewer errors than expected, this can also be taken as a positive sign. It indicates that implementing certain security measures early in the development lifecycle will go a long way in reducing unexpected errors during testing.
- **No Critical Defects:** After drawing these conclusions, project managers should ensure that all critical defects found during testing are resolved within one month after launch. This allows them time to notify users about any lingering issues and rectify any critical bugs before releasing the final copy to end users. Doing so will increase the likelihood of satisfied users and increase early adopter interest in your product.

Agile Metrics in Practice-

Practice	Aim	Based on	Beneficiary	When
Software size [6]	Estimation of size/effort	User stories	Project Manager	At start of each iteration
Velocity [6]	Productivity of team	User stories	Project Manager	At end of each iteration
Burndown [6]	Progress monitoring	User stories	Team	At end of each iteration
Cumulative Flow [6]	Observation of lead time and WIP queue depth	Work in progress	Top managers/ customers	At end of each iteration
Responding to change [6]	Ability of team to handover quality	Defects fixing cost	Project Manager	At end of each itera-tion/project
Earned business value [6]	Monitoring bv delivered to customer	Business value	Top managers/ customers	As each feature is delivered
Total estimation effort [6]	Planning and budgeting	User stories and reworks	Top managers/ customers	At beginning of project

Story estimation	Time spent on story estimation	User stories	Team	Beginning of iteration
Requirements ambiguity	Misinterpreted requirements	User stories	Team	End of iteration
Unfinished stories	Identify problems with stories	User stories	Team	End of iteration
Number of impediments	Identify impediments team faced	Impediments identified in retrospec-tives	Team	During retro-spectives
Work-in-progress	Items being actively worked on	User stories on kanban board	Team	During iteration
Unplanned changes	Changes able to process in increment	Changes to user stories	Team	During increment
Employee satisfaction	Individual satisfaction	Questionnaires/ Surveys	Individuals	End of iteration

Software Metrics and Tools-

Software Metrics	Usage in ASD	Measurement Tools
Customer satisfaction	Measures the outcome of customer satisfaction.	Customer feedback, Net promoter score
Business value delivered	Helps the product managers to link their products with their business value.	Customer feedback
On-time delivery	Helps in measuring customer expectations regarding the timely delivery of the product.	Burn-down charts, Burn-up charts
Quality	Measures the conformance of the project with the quality standards	Number of defects
Productivity	Helps managers to understand how much the team is delivering.	Burn-up charts
Predictability	Measures the amount of work the team has to perform to complete all the planned work for the successful delivery of the product.	Velocity trend charts
Process improvement	Helps the project managers understand how the proposed adjustments are impacting productivity.	Cumulative flow charts
Project visibility	Helps in sharing project updates and progress with all stakeholders.	Dependency charts
Project scope	Helps the project manager and their teams to set and accomplish goals over a specific period.	Burn-down charts, Kanban boards
Budget versus actual cost	Helps the project manager to compare the budget with the actual cost of the project and forecast revenues and expenses.	Cumulative flow charts, Dependency charts

Software Metrics and their Organizational Level

Software Metrics	Description	Organizational Level
Story point estimation	Measures the outcome of customer satisfaction.	Team
Velocity	Helps the product managers to link their products with their business value.	Team
Sprint burn-down	Helps in measuring customer expectations regarding the timely delivery of the product.	Program
Defects in production	Measures the conformance of the project with the quality standards	Team
Planned velocity	Helps managers to understand how much the team is delivering.	Program
Program velocity per sprint	Measures the amount of work the team has to perform to complete all the planned work for the successful delivery of the product.	Program
Number of features per Program Increment (PI)	Helps the project managers understand how the proposed adjustments are impacting productivity.	Program
Planned program velocity per sprint	Helps in sharing project updates and progress with all stakeholders.	Program
Release burn-down	Helps the project manager and their teams to set and accomplish goals over a specific period.	Program
Test Coverage	Helps the project manager to compare the budget with the actual cost of the project and forecast	Program

Software Quality Attributes and their metrics-

Software Quality Attribute	Quality Metrics	Description
Maintainability	Mean time to repair (MTTR)	Maintainability is measured by MTTR. MTTR is the average time to repair a system technically or mechanically. It includes both the repair time and any testing time.
Availability	Mean time to failure (MTTF)	Availability is measured as a relation of MTTF and MTTR.
	Mean time to repair (MTTR)	
Reliability	Mean time to failure (MTTF)	Reliability is measured as the inverse of MTTF
Portability	Number of successful ports	Availability is measured as a relation of "Number of successful ports" and "Total number of ports.
	Total number of ports	
Testability	Observability	Testability is measured by observability, simplicity, modularity, and stability.
	Simplicity	
	Modularity	
	Stability	
Reusability	Weighted Maintainability (Product of Weight and Maintainability metric)	Reusability is measured as the summation of weight multiplied by the composite metric of each attribute defined for measuring reusability. Maintainability, adaptability, documentation, complexity, and availability are the attributes that define reusability.
	Weighted Adaptability (Product of Weight and Adaptability metric)	
	Weighted Documentation (Product of Weight and Documentation metric)	
	Weighted Complexity (Product of Weight and Complexity metric)	
	Weighted Availability (Product of Weight and Availability metric)	

Estimation and Project Variables-

Estimation: Schedule, Scope, Cost, and Effort are the four major variables that typically control Software projects. Any changes in any of these variables can have an effect on a project. Estimation is a process to forecast these variables to develop or maintain software based on the information specified by the client.

There are three main challenges faced during estimation i.e., Uncertainty, Self-knowledge, and Consistency of Method used for Estimation. Usage of standardized and scientific estimation methods for estimating size, effort, and schedule, helps towards maintaining minimal variance between the planned estimates and actual values thereby achieving maximum estimation accuracy. This provides a better client experience. All estimation needs for a project cannot be determined by a single method. It is important to have different methods of estimation for different stages.

Planning and Estimation in Agile projects bring a lot of focus on preparation and forecasting. Both these activities are done keeping business context in mind and measurable value delivery is committed to the client. Therefore, it is recommended to have required planning and estimation in Agile from the start of the project, in order to ensure better risk coverage and higher predictability.

During the Project Initiation stage, functional requirements are at a high level, or the details of requirements are not well-formulated and documented. In the Product Backlog, many requirements are still at the Epic or Feature level. At this point, estimation is required as it will help in the prioritization and planning of the Releases. Different estimation techniques can be used at this stage. Some of them are

- QFPA (Quick Function Point Analysis)
- Use Case Points
- Complexity based estimation
- COCOMO (Constructive Cost Model)
- COSMIC FP

During the Project Initiation stage, estimation should be done as a group activity, where results should be compared and disagreements are resolved. All the project assumptions and risks should be highlighted clearly. Typically, the team for performing this activity will include:

- Product Owner
- Clients & Business Stakeholders
- Project Manager
- Developers and Designers
- Testers